Day 17: Kubernetes Services & Networking

COVER SLIDE / OVERVIEW

```
dhiru@docker-desktop:~/day17-services

# ClusterIP, CoreDNS, EndpointSlices & Selector Breaking
$ kubectl apply -f svc-clusterip.yaml
$ kubectl run tmp-rm -it --image=busybox:1.36 -- nslookup hello
$ kubectl patch service hello --type=merge -p '{"spec":{"selector":
{"app":"wrong-app"}}}'
* Status: ClusterIP, DNS & Self-Healing Verified
```

MISSION ROADMAP (01 TO 08)

- ✓ **01: Environment & Context Inspection**
- ✓ **02: 3-Replica Workload Deployment**
- ✓ **03: ClusterIP Service & EndpointSlices**
- ✓ **04: CoreDNS Name Resolution & Load Balancing**
- ✓ **05: Pod IP & Endpoint Mapping**
- ✓ **06: Pod Termination & Dynamic Endpoint Healing**
- ✓ **07: Selector Corruption & Self-Healing Fix**
- ✓ **08: Interview Readiness & Architecture Q&A**

01 / Core Concepts & Service Commands

Service & Networking Concepts

- **ClusterIP:** The default virtual IP exposing a Service internally within the cluster.
- **Selector:** Label query (e.g., `app: hello`) used to route traffic to target Pods.
- **EndpointSlice:** Scalable Kubernetes resource tracking active backend Pod IPs and ports.
- **CoreDNS:** In-cluster DNS service enabling name lookup (e.g., `hello.default.svc.cluster.local`).
- **port vs targetPort:** `port` is exposed by the Service; `targetPort` is listened to by the Pod container.

Essential kubectl Service Commands

- `kubectl get svc -o wide`
- `kubectl describe svc hello`
- `kubectl get endpointslices -l kubernetes.io/service-name=hello`
- `kubectl get pods -l app=hello -o wide`
- `kubectl patch service hello --type=merge -p '{"spec":{"selector": {"app":"wrong-app"}}}'`

02 / Step 1 — Environment & Cluster Verification

Step 1: Verified cluster context (`docker-desktop`), confirmed single-node status (`v1.36.1`), and checked baseline deployments and services.

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl config
current-context
docker-desktop
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get nod
es
NAME             STATUS   ROLES           AGE     VERSION
docker-desktop   Ready    control-plane   5h39m   v1.36.1
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get dep
loyments
No resources found in default namespace.
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get ser
vices
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   5h39m
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

03 / Step 2 — 3-Replica agnhost Workload Deployment

Step 2: Deployed `hello` workload with 3 replicas using `agnhost` on port 8080 and verified `app=hello` labels.

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl create
deployment hello --replicas=3 --image=registry.k8s.io/e2e-test-images/agnhost:2.39 -- //agnhost netexec --http-port=8080
deployment.apps/hello created
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get dep
loyment hello
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
hello   3/3     3            3           10s
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get pod
s -l app=hello --show-labels
NAME                     READY   STATUS    RESTARTS   AGE   LABELS
hello-756774f8c5-6q8sg   1/1     Running   0          18s   app=hello,pod-template-hash=756774f8c5
hello-756774f8c5-7whzm   1/1     Running   0          18s   app=hello,pod-template-hash=756774f8c5
hello-756774f8c5-rzvj2   1/1     Running   0          18s   app=hello,pod-template-hash=756774f8c5
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

04 / Step 3 — ClusterIP Creation & EndpointSlices

Step 3: Applied `svc-clusterip.yaml` manifest, exposed port 80 mapped to targetPort 8080, and verified EndpointSlices.

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> cat .\svc-clusterip.yaml
apiVersion: v1
kind: Service
metadata:
  name: hello
spec:
  type: ClusterIP
  selector:
    app: hello          # 'app' goes inside selector
  ports:
    - protocol: TCP
      port: 80          # The port the service exposes
      targetPort: 8080  # 'targetPort' goes inside ports list
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl apply -f .\svc-clusterip.yaml
service/hello created
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get svc hello -o wide
NAME    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE   SELECTOR
hello   ClusterIP   10.111.166.197   <none>        80/TCP    6s    app=hello
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl describe svc hello
Name:                     hello
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=hello
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.111.166.197
IPs:                      10.111.166.197
Port:                     <unset>  80/TCP
TargetPort:               8080/TCP
Endpoints:                10.1.0.26:8080,10.1.0.27:8080,10.1.0.25:8080
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get endpointslices -l kubernetes.io/service-name=hello
NAME          ADDRESSTYPE   PORTS   ENDPOINTS                     AGE
hello-hk67t   IPv4          8080    10.1.0.26,10.1.0.27,10.1.0.25   46s
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

05 / Step 4 — CoreDNS Resolution & Load Balancing

Step 4: Executed `nslookup hello` and a 6-request `wget` loop from a temporary BusyBox Pod, demonstrating HTTP load balancing across backend replicas.

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl run tmp-rm --image=bu
sybox:1.36 --rm -it --restart=Never -- sh -c "nslookup hello; nslookup hello.default.svc.cluster.local; for i in 1 2 3 4 5 6; do wget -qO- http://hello/hostname; echo '';
 done"
Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find hello.cluster.local: NXDOMAIN

** server can't find hello.svc.cluster.local: NXDOMAIN

Name:   hello.default.svc.cluster.local
Address: 10.111.166.197

** server can't find hello.cluster.local: NXDOMAIN

** server can't find hello.svc.cluster.local: NXDOMAIN


Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   hello.default.svc.cluster.local
Address: 10.111.166.197


hello-756774f8c5-rzvj2
hello-756774f8c5-7whzm
hello-756774f8c5-7whzm
hello-756774f8c5-rzvj2
hello-756774f8c5-6q8sg
hello-756774f8c5-rzvj2
```

06 / Step 5 — Backend Endpoint IP Mapping

Step 5: Inspected EndpointSlices and cross-referenced IP addresses directly with active Pod IPs (`10.1.0.25`, `10.1.0.26`, `10.1.0.27`).

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get endpointslices -l
 kubernetes.io/service-name=hello -o wide
NAME          ADDRESSTYPE   PORTS   ENDPOINTS                     AGE
hello-hk67t   IPv4          8080    10.1.0.26,10.1.0.27,10.1.0.25   17m
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get pods -l app=hello
 -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP          NODE             NOMINATED NODE   READINESS GATES
hello-756774f8c5-6q8sg   1/1     Running   0          27m   10.1.0.26   docker-desktop   <none>           <none>
hello-756774f8c5-7whzm   1/1     Running   0          27m   10.1.0.27   docker-desktop   <none>           <none>
hello-756774f8c5-rzvj2   1/1     Running   0          27m   10.1.0.25   docker-desktop   <none>           <none>
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

07 / Step 6 — Pod Termination & Self-Healing Endpoints

Step 6: Deleted backend pod `hello-756774f8c5-6q8sg` and confirmed automatic endpoint updates with replacement pod `w4n5z` (`10.1.0.33`).

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get pods -l app=hello

NAME                     READY   STATUS    RESTARTS   AGE
hello-756774f8c5-6q8sg   1/1     Running   0          29m
hello-756774f8c5-7whzm   1/1     Running   0          29m
hello-756774f8c5-rzvj2   1/1     Running   0          29m
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl delete pod hello-756774f8c5-6q8sg
pod "hello-756774f8c5-6q8sg" deleted from default namespace
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get endpointslices -l kubernetes.io/ser
vice-name=hello -o wide
NAME          ADDRESSTYPE   PORTS   ENDPOINTS                       AGE
hello-hk67t   IPv4          8080    10.1.0.27,10.1.0.25,10.1.0.33   19m
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get pods -l app=hello

NAME                     READY   STATUS    RESTARTS   AGE
hello-756774f8c5-7whzm   1/1     Running   0          30m
hello-756774f8c5-rzvj2   1/1     Running   0          30m
hello-756774f8c5-w4n5z   1/1     Running   0          31s
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

08 / Step 7 — Selector Corruption (`app=wrong-app`)

Step 7: Patched selector to `wrong-app`, demonstrating complete loss of backend endpoints while ClusterIP remains active.

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl patch service hello --type=merge -p '{\"spec\":{\"selector\
":{\"app\":\"wrong-app\"}}}'
service/hello patched
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl describe service hello
Name:                     hello
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=wrong-app
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.111.166.197
IPs:                      10.111.166.197
Port:                     <unset>  80/TCP
TargetPort:               8080/TCP
Endpoints:
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get endpointslices -l kubernetes.io/service-name=hello
NAME          ADDRESSTYPE   PORTS     ENDPOINTS   AGE
hello-hk67t   IPv4          <unset>   <unset>     22m
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

09 / Step 8 — Selector Restoration (`app=hello`)

Step 8: Restored Service selector back to `app=hello`, immediately reconnecting all 3 backend endpoints.

```
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl patch service hello --type=merge -p '{\"spec\":{\"selector\
":{\"app\":\"hello\"}}}'
service/hello patched
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl get endpointslices -l kubernetes.io/service-name=hello -o w
ide
NAME          ADDRESSTYPE   PORTS   ENDPOINTS                     AGE
hello-hk67t   IPv4          8080    10.1.0.25,10.1.0.27,10.1.0.33   29m
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc> kubectl describe service hello
Name:                     hello
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=hello
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.111.166.197
IPs:                      10.111.166.197
Port:                     <unset>  80/TCP
TargetPort:               8080/TCP
Endpoints:                10.1.0.25:8080,10.1.0.27:8080,10.1.0.33:8080
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
PS C:\Users\dhiru\Desktop\New folder\90 Days DevOps Winter Arc\Week-3-Kubernetes\Day-17\day17-kubernetes-services-90-days-DevOps-winter-arc>
```

10 / Section 07 — Troubleshooting Matrix & Review Checklist

Problem / Scenario	Diagnostic Command	Resolution & Key Learning
No Endpoints Available	`kubectl describe svc hello`	Check Service selector against Pod labels using `kubectl get pods --show-labels`.
DNS Lookup Fails	`nslookup <service-name>`	Verify Service existence, correct namespace context, and CoreDNS Pod status in `kube-system`.
DNS Resolves but Request Times Out	`kubectl get endpointslices`	Verify `targetPort` matches application container listening port (8080).
Uneven Load Balancing	`wget -qO- http://hello/hostname`	Kubernetes kube-proxy uses random iptables/IPVS rules; exact round-robin per request is not guaranteed.

11 / Section 08 — Interview Readiness Architecture Q&A

1. Why use a Service instead of Pod IPs?

- Pod IPs are ephemeral and change upon restart/reschedule. Services provide a permanent virtual IP and DNS record.

2. Difference between port and targetPort?

- `port` is the frontend port exposed by the Service; `targetPort` is the backend port open on the container.

3. How does CoreDNS resolve Service names?

- CoreDNS maps internal names (`hello.default.svc.cluster.local`) directly to the Service ClusterIP address.

4. What is an EndpointSlice?

- A scalable Kubernetes object tracking IP endpoints for a Service, replacing legacy monlithic Endpoint objects.